FRED TALKS

19th April 2026

CONTENTS

The Night Watch

JAMES MICKENS · 11 APR 2026 · SOURCE

The Night Watch

J A M E S M I C K E N S

James Mickens is a researcher in the Distributed Systems group at Microsoft's Redmond lab. His current research focuses on web applications,

with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters.

James received his PhD in computer science from the University of Michigan, and a bachelor's degree in computer science from Georgia Tech. mickens@microsoft.com

s a highly trained academic researcher, I spend a lot of time trying to advance the frontiers of human knowledge. However, as someone who was born in the South, I secretly believe that true progress is

A

a fantasy, and that I need to prepare for the end times, and for the chickens coming home to roost, and fast zombies, and slow zombies, and the polite zombies who say "sir" and "ma'am" but then try to eat your brain to acquire your skills. When the revolution comes, I need to be prepared; thus, in the quiet moments, when I'm not producing incredible scientific breakthroughs, I think about what I'll do when the weather forecast inevitably becomes

RIVERS OF BLOOD ALL DAY EVERY DAY. The main thing that I ponder is who will be in my gang, because the likelihood of post-apocalyptic survival is directly related to the size and quality of your rag-tag group of associates. There are some obvious people who I'll need to recruit: a locksmith (to open doors); a demolitions expert (for when the locksmith has run out of ideas); and a person who can procure, train, and then throw snakes at my enemies

(because, in a world without hope, snake throwing is a reasonable way to resolve disputes). All of these people will play a role in my ultimate success as a dystopian warlord philosopher. However, the most important person in my gang will be a systems programmer. A person who can debug a device driver or a distributed system is a person who can be trusted in a Hobbesian nightmare of breathtaking scope; a systems programmer has seen the terrors of the world and understood the intrinsic horror of existence. The systems programmer has written drivers for buggy devices whose firmware was implemented by a drunken child or a sober goldfish. The systems program- mer has traced a network problem across eight machines, three time zones, and a brief diversion into Amish country, where the problem was transmitted in the front left hoof of a mule named Deliverance. The systems program- mer has read the kernel source, to better understand the deep ways of the universe, and the systems programmer has seen the comment in the sched- uler that says "DOES THIS WORK LOL," and the systems programmer has wept instead of LOLed, and the systems programmer has submitted a kernel patch to restore balance to The Force and fix the priority inversion that was causing MySQL to hang. A systems programmer will know what to do when society breaks down, because the systems programmer already lives in a world without law.

Listen: I'm not saying that other kinds of computer people

are useless. I believe (but cannot prove) that PHP developers have souls. I think it's great that database people keep trying to improve select-from-where, even though the only queries that cannot be expressed using select-from-where are inap- propriate limericks from "The Canterbury Tales." In some way that I don't yet understand, I'm glad that theorists are investigating the equivalence between five-dimensional Tur- ing machines and Edward Scissorhands. In most situations, GUI designers should not be forced to fight each other with tridents and nets as I yell "THERE ARE NO MODAL DIA- LOGS IN SPARTA." I am like the Statue of Liberty: I accept everyone, even the wretched and the huddled and people who enjoy Haskell. But when things get tough, I need mission-crit- ical people; I need a person who can wear night-vision goggles and descend from a helicopter on ropes and do classified things to protect my freedom while country music plays in the background. A systems person can do that. I can realistically give a kernel hacker a nickname like "Diamondback" or "Zeus Hammer." In contrast, no one has ever said, "These semi-

transparent icons are really semi-transparent! IS THIS THE WORK OF ZEUS HAMMER?”

I picked that last example at random. You must believe me when I say that I have the utmost respect for HCI people.

However, when HCI people debug their code, it’s like an

art show or a meeting of the United Nations. There are tea breaks and witticisms exchanged in French; wearing a non- functional scarf is optional, but encouraged. When HCI code doesn’t work, the problem can be resolved using grand theo- ries that relate form and perception to your deeply personal feelings about ovals. There will be rich debates about the socioeconomic implications of Helvetica Light, and at some point, you will have to decide whether serifs are daring state- ments of modernity, or tools of hegemonic oppression that implicitly support feudalism and illiteracy. Is pinching-and- dragging less elegant than circling-and-lightly-caressing?

These urgent mysteries will not solve themselves. And yet, after a long day of debugging HCI code, there is always hope, and there is no true anger; even if you fear that your drop- down list should be a radio button, the drop-down list will suffice until tomorrow, when the sun will rise, glorious and

vibrant, and inspire you to combine scroll bars and left-click- ing in poignant ways that you will commemorate in a sonnet when you return from your local farmer’s market.

This is not the world of the systems hacker. When you debug a distributed system or an OS kernel, you do it Texas-style. You gather some mean, stoic people, people who have seen things die, and you get some primitive tools, like a compass and a rucksack and a stick that’s pointed on one end, and you walk into the wilderness and you look for trouble, possibly while

using chewing tobacco. As a systems hacker, you must be pre- pared to do savage things, unspeakable things, to kill runaway threads with your bare hands, to write directly to network ports using telnet and an old copy of an RFC that you found in the Vatican. When you debug systems code, there are no high- level debates about font choices and the best kind of turquoise, because this is the Old Testament, an angry and monochro- matic world, and it doesn’t matter whether your Arial is Bold or Condensed when people are covered in boils and pestilence and Egyptian pharaoh oppression. HCI people discover bugs by receiving a concerned email from their therapist. Systems people discover bugs by waking up and discovering that their first-born children are missing and “ETIMEDOUT ” has been written in blood on the wall.

What is despair? I have known it—hear my song. Despair is when you're debugging a kernel driver and you look at a mem- ory dump and you see that a pointer has a value of 7. THERE IS NO HARDWARE ARCHITECTURE THAT IS ALIGNED ON

7. Furthermore, 7 IS TOO SMALL AND ONLY EVIL CODE WOULD TRY TO ACCESS SMALL NUMBER MEMORY.

Misaligned, small-number memory accesses have stolen decades from my life. The only things worse than misaligned, small-number memory accesses are accesses with aligned buf- fer pointers, but impossibly large buffer lengths. Nothing ruins a Friday at 5 P.M. faster than taking one last pass through the log file and discovering a word-aligned buffer address, but a buffer length of NUMBER OF ELECTRONS IN THE UNI-

VERSE. This is a sorrow that lingers, because a 2893 byte read is the only thing that both Republicans and Democrats agree is wrong. It's like, maybe Medicare is a good idea, maybe not, but there's no way to justify reading everything that ever existed a jillion times into a mega-jillion sized array. This constant war on happiness is what non-systems people do not understand about the systems world. I mean, when a machine learning

algorithm mistakenly identifies a cat as an elephant, this is

actually hilarious. You can print a picture of a cat wearing an elephant costume and add an ironic caption that will entertain people who have middling intellects, and you can hand out cop- ies of the photo at work and rejoice in the fact that everything is still fundamentally okay. There is nothing funny to print when you have a misaligned memory access, because your machine is dead and there are no printers in the spirit world.

An impossibly large buffer error is even worse, because these errors often linger in the background, quietly overwriting your state with evil; if a misaligned memory access is like a criminal burning down your house in a fail-stop manner, an impossibly large buffer error is like a criminal who breaks into your house, sprinkles sand atop random bedsheets and toothbrushes, and then waits for you to slowly discover that your world has been tainted by madness. Indeed, the common discovery mode for an impossibly large buffer error is that your program seems to

be working fine, and then it tries to display a string that should say "Hello world," but instead it prints "#a[5]:3!" or another syntactically correct Perl script, and you're like WHAT THE HOW THE, and then you realize that your prodigal memory accesses have been stomping around the heap like the Incred- ible Hulk when asked to write an essay entitled "Smashing Considered Harmful."

You might ask, “Why would someone write code in a grotesque language that exposes raw memory addresses? Why not use

a modern language with garbage collection and functional programming and free massages after lunch?” Here’s the answer: Pointers are real. They’re what the hardware under- stands. Somebody has to deal with them. You can’t just place a LISP book on top of an x86 chip and hope that the hardware learns about lambda calculus by osmosis. Denying the exis- tence of pointers is like living in ancient Greece and denying the existence of Krackens and then being confused about why none of your ships ever make it to Morocco, or Ur-Morocco,

or whatever Morocco was called back then. Pointers are like Krackens—real, living things that must be dealt with so that polite society can exist. Make no mistake, I don’t want to write systems software in a language like C++. Similar to the Necro- nomicon, a C++ source code file is a wicked, obscure document that’s filled with cryptic incantations and forbidden knowl- edge. When it’s 3 A.M., and you’ve been debugging for 12 hours, and you encounter a virtual static friend protected volatile

templated function pointer, you want to go into hibernation and awake as a werewolf and then find the people who wrote the C++ standard and bring ruin to the things that they love. The C++ STL, with its dyslexia-inducing syntax blizzard of colons and angle brackets, guarantees that if you try to declare any reasonable data structure, your first seven attempts will result in compiler errors of Wagnerian fierceness:

Syntax error: unmatched thing in thing from std::nonstd::_map<_Cyrillic, _$$$dollars>const basic_string< epic_ mystery,mongoose_traits < char>, _default_alloc_<casual_ Fridays = maybe>>

One time I tried to create a list<map<int>>, and my syntax errors caused the dead to walk among the living. Such things are clearly unfortunate. Thus, I fully support high-level languages in which pointers are hidden and types are strong and the declaration of data structures does not require you to solve a syntactical puzzle generated by a malevolent extraterrestrial species. That being said, if you find yourself drinking a martini and writing programs in garbage-collected, object-oriented Esperanto, be aware that the only reason that the Esperanto runtime works is because there are systems people who have exchanged any hope of losing their virginity for the exciting opportunity to think about hex numbers and their relationships

with the operating system, the hardware, and ancient blood rituals that Bjarne Stroustrup performed at Stonehenge.

Perhaps the worst thing about being a systems person is that other, non-systems people think that they understand the daily tragedies that compose your life. For example, a few weeks ago, I was debugging a new network file system that my research group created. The bug was inside a kernel-mode component, so my machines were crashing in spectacular and vindic-

tive ways. After a few days of manually rebooting servers, I had transformed into a shambling, broken man, kind of like a computer scientist version of Saddam Hussein when he was pulled from his bunker, all scraggly beard and dead eyes and florid, nonsensical ramblings about semi-imagined enemies.

As I paced the hallways, muttering Nixonian rants about my code, one of my colleagues from the HCI group asked me what my problem was. I described the bug, which involved concurrent threads and corrupted state and asynchronous message delivery across multiple machines, and my coworker said,

“Yeah, that sounds bad. Have you checked the log files for errors?” I said, “Indeed, I would do that if I hadn’t broken every component that a logging system needs to log data. I have a network file system, and I have broken the network, and I have broken the file system, and my machines crash when I make eye contact with them. I HAVE NO TOOLS BECAUSE I’VE DESTROYED MY TOOLS WITH MY TOOLS. My only logging

option is to hire monks to transcribe the subjective experience of watching my machines die as I weep tears of blood.” My co- worker, in an earnest attempt to sympathize, recounted one of his personal debugging stories, a story that essentially involved an addition operation that had been mistakenly replaced with

a multiplication operation. I listened to this story, and I said, “Look, I get it. Multiplication is not addition. This has been known for years. However, multiplication and addition are at least related. Multiplication is like addition, but with more addition. Multiplication is a grown-up pterodactyl, and addi- tion is a baby pterodactyl. Thus, in your debugging story, your code is wayward, but it basically has the right idea. In contrast, there is no family-friendly GRE analogy that relates what my code should do, and what it is actually doing. I had the mod-

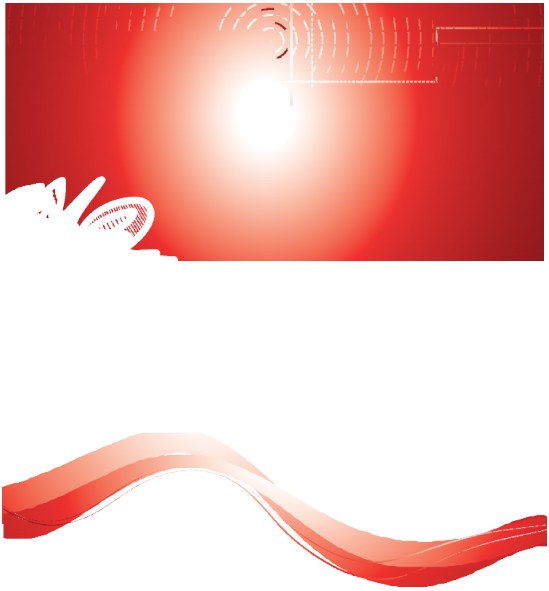
est goal of translating a file read into a network operation, and now my machines have tuberculosis and orifice containment issues. Do you see the difference between our lives? When you asked a girl to the prom, you discovered that her father was a cop. When I asked a girl to the prom, I DISCOVERED THAT HER FATHER WAS STALIN.”

In conclusion, I’m not saying that everyone should be a systems hacker. GUIs are useful. Spell-checkers are useful. I’m glad that people are working on new kinds of bouncing icons because they believe that humanity has solved cancer and homelessness and now lives in a consequence-free world

of immersive sprites. That’s exciting, and I wish that I could join those people in the 27th century. But I live here, and I live now, and in my neighborhood, people are dying in the streets. It’s like, French is a great idea, but nobody is going to invent French if they’re constantly being attacked by bears. Do you see? SYSTEMS HACKERS SOLVE THE BEAR MENACE.

Only through the constant vigilance of my people do you get

the freedom to think about croissants and subtle puns involv- ing the true father of Louis XIV. So, if you see me wandering the halls, trying to explain synchronization bugs to confused monks, rest assured that every day, in every way, it gets a little better. For you, not me. I’ll always be furious at the number 7, but such is the hero’s journey.



usenix
THE ADVANCED
COMPUTING SYSTEMS
ASSOCIATION

Do you know about the USENIX Open Access Policy?

USENIX is the first computing association to offer free and open access to all of our conferences proceedings and videos. We stand by our mission to foster excellence and innovation while supporting research with a practical bias. Your membership fees play a major role in making this endeavor successful.

Please help us support open access.

Renew your USENIX membership

and ask your colleagues to join or renew today!

www.usenix.org/membership

Joy & Curiosity #82

THORSTEN BALL · 19 APR 2026 · SOURCE

This one's short, because it's been a week full of programming and building, less reading. And this weekend's equally busy, so here's a question I've been flipping around in my head for months now:

What are we learning about working with these models that will be valuable in the future?

In his Lex Fridman interview, ThePrimeagen said something that stuck with me: “Is anyone actually falling behind for not using AI then? Because if the interface is going to change so greatly that all of your habits need to fundamentally change […], have I actually fallen behind at all? Or will the next gen actually just be so different from the current one that it’s like, yeah, you’re over there actually doing punch card AI right now. I’m going to come in at compiler time AI, so different that it’s like what’s a punch card?”

There’s something to this. The frontier models are now much more forgiving when it comes to prompts. We no longer have to write “you are a senior engineer” in our prompts. “Don’t make mistakes” is more a prayer than a helpful trick. The days of the Prompt Engineer won’t be visible on the timeline if we zoom out to even five years.

Nowadays, I’m even convinced that a lot of what we considered important for manual context management is now no longer needed. (Yes, we’re shipping soon.) We’re close to the point where you no longer have to care whether you’re at 30% or 70% of the context window.

And I’m also convinced that the models will get even better.

Now, maybe it is a form of sunk cost fallacy, a bias talking, but still: I do think that I got better at working with these models over the past two years. It might not be relevant anymore whether I write down my task before or after I include a file in a prompt, but I think I’ve gained some meta-abilities that made me better at solving problems through the use of agents: chopping up problems into engineering tasks and sequencing them, figuring out what the pitfalls (that wouldn’t be pitfalls for humans) are, knowing what’s poison in the codebase and what isn’t. Stuff like that.

In the most general sense, I think I’ve learned how to work with artificial intelligence. And if prompt engineering tricks are punch cards, then that might be seen as learning about computation.

- This is, at least for me, already a Hall of Fame comment: “For reasons which it would take a while to unpack, if is often the case that the best (or sometimes only) way to find out what programming actually needs to be done, is to program something that’s not it, and then replace it. This may need to be done multiple times. Programming is only occasionally the final product, it is much more often the means of working through what it is that is actually needed. This is very difficult for the people who ask for the software, to understand, and it is quite often very difficult for the people doing the programming to understand. Most of what is being done, during programming, is working through the problem space in a way which

will make it more obvious what your mistakes are, in your understanding of the problem and what a solution would look like. Once you have arrived at that understanding, then there are a variety of ways to make what you need, but that is not the rate-limiting step." So, so, so good. This is what software development is: learning.

- Fractal Paris and Fractal Istanbul. Lovely!

- Rands: The Complicators, The Drama Aggregators, and The Avoiders. Read and recognize people you've worked with.

- Brian Cantrill on the peril of laziness lost: "The problem is that LLMs inherently lack the virtue of laziness. Work costs nothing to an LLM. LLMs do not feel a need to optimize for their own (or anyone's) future time, and will happily dump more and more onto a layercake of garbage. Left unchecked, LLMs will make systems larger, not better." Read this at the start of the week and then constantly thought of it whenever I asked my agent whether this is "truly the simplest, most minimal, as-little-as-possible and as-much-as-needed solution?"

- Vicki Boykis, in some sense in harmony with Brian Cantrill's thoughts, on Mechanical Sympathy: "Mechanical sympathy for both developers and end-users means understanding when asyncio is and is not helpful. It means using the right language, the right build system, the right font. It means using the least amount of tooling possible. Allowing for local development. It means reading code inside out rather than top to bottom. Using uv. Removing code where not necessary. Respecting boundaries."

- stevey wrote a tweet about AI adoption at Google and got pushback from Demis Hassabis and others and, well, I actually don't care *that much* about AI adoption at Google, but I find this one thought in there very fascinating: "There has been an industry-wide hiring freeze for 18+ months, during which time nobody has been moving jobs. So there are no clued-in people coming in from the outside to tell Google how far behind they are, how utterly mediocre they have become as an eng org." I know that people aren't sure whether there are more or less software jobs right now, but from where I'm sitting it does look like hiring has slowed and I find it fascinating to think about the second-order effects of that: is there less industry-wide diffusion of frontier knowledge because hiring has slowed?

- Shifted something in my brain: Nucleus Nouns. Very good and much more thought-inspiring than the usual "focus! focus! focus!" chants.

- The Closing of the Frontier: "There is something special about training a model on all of humanity's data and then locking it up for the benefit of a few well-connected organizations that you have relationships with. Maybe you'll notice another historical pattern here. Extract

value from a population that can't meaningfully consent, concentrate the returns within a small inner circle, and then offer some version of charity to the people you extracted from as moral cover for the arrangement."

- Andy Matuschak has the Practice Guide for Computer printed out and hanging above his desk.

- Apparently I'm the last person to learn about this idea, but who cares, it's great and I think I want to try this: The Spark File.

- Sometimes I read things online and it makes me really happy that we have the Internet and that smart, beautiful minds share their thoughts online. Here's James Somers with his idea of the Paper Computer: "Now that we have actually good AI, I have this vision of a form of computing that doesn't involve me using a computer so much. Imagine you had the day's emails to go through. It would be nice if the ones that required a simple decision could be dispatched with a few pen-strokes: I could write down a date that would work for that meeting; check a box to accept that invitation; etc. If an email required me to review a draft, I'd love to mark up a print version on my couch, sans screen, and have those notes scanned and sent off as if I'd done the whole thing on Google Docs."

- Tim Zaman, who worked at NVIDIA, Tesla, X, Google DeepMind and now at OpenAI on AI infrastructure on Getting Into AI Infra. I'm *convinced* that posts like these create and change entire lifes. I love it. Also: nearly made me want to build a cluster.

- It's been a while since I've thought about people who have *not* yet walked through the one-way door that makes you say "holy shit, AI is going to change everything", but Armin shared his thoughts after encountering people still being skeptical: The Center Has a Bias. Well worth reading.

- Dwarkesh Patel shared what he learned this week and note how interesting that is and how enjoyable it is to read, even though (or is it because of?) it's not polished at all.

- Drew Breunig, following the Anthropic Mythos frenzy and some companies closing their open-source projects down for fear of security vulnerabilities being discovered, says Cybersecurity Looks Like Proof of Work Now: "If Mythos continues to find exploits so long as you keep throwing money at it, security is reduced to a brutally simple equation: to harden a system you need to spend more tokens discovering exploits than attackers will spend exploiting them."

- But antirez disagrees: AI cybersecurity is not proof of work. Both posts are very interesting and I recommend reading through them.

- I'm in the process of setting up my 2013 MacBook Pro for my 4-year-old daughter and Peter recommended this lovely page to let her type on: tiny-terminal.com.

- So, of course, I had to fork it, bought a domain, and let Amp translate it to German so my kids can type words they already know in there: kleines-terminal.de.

- Turing Award winner Michael Rabin has passed away. Here's "an assorted collection of quotations due to Professor Michael Rabin, produced at Harvard University during the Fall 1997 incarnation of the course Computer Science 226r": Rabinism Collection.

- Thoughts and Feelings around Claude Design.

- Another way to think about the question of whether AI will create more jobs or not, by Aaron Levie: "Why will AI create more jobs in plenty of industries? It's because we're going to use AI to accelerate output in one area, and then eventually you run into a new bottleneck somewhere else in the process that still requires humans." This sounds very likely to me. But, of course, "more jobs" doesn't mean it'll be the same jobs and then some. Everything's changing.

- Related, Gary Bernhardt: "This might be a Mel moment. It's not immediately obvious that Mel is a tragic story. He clearly loved the work. Then the work changed and, presumably, he was left behind. The thing he perfected no longer mattered. There might be millions of Mels right now."

- Wow, look at just the table of contents here: "I have for years been interested in sleep research due to my professional involvement in memory and learning. This article attempts to produce a synthesis of what is known about sleep with a view to practical applications, esp. in people who need top-quality sleep for their learning or creative achievements."

[Daily article] April 19: 1986 World Snooker Championship

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 19 APR 2026 · SOURCE

The 1986 World Snooker Championship was a professional snooker tournament that took place between 19 April and 5 May 1986 at the Crucible Theatre in Sheffield, England. It was the sixth and final ranking event of the 1985–86 snooker season and the 1986 edition of the World

Snooker Championship, which was first held in 1927. The total prize fund was £350,000, with £70,000 awarded to the winner. The defending champion was Dennis Taylor, who had defeated Steve Davis 18–17 in the 1985 World Snooker Championship final to win his first world title. Taylor lost in the first round of the 1986 event 6–10 to Mike Hallett. Joe Johnson (pictured), the world number 16, defeated Davis 18–12 in the final to win his sole ranking event. Prior to the competition, the bookmakers' odds for a Johnson victory were 150/1. There were a total of 20 century breaks compiled during the tournament, the highest of which was a 134 made by Davis. (Full article...).

Read more: https://en.wikipedia.org/wiki/1986_World_Snooker_Championship

---

Today's selected anniversaries:

1861:

The first bloodshed of the American Civil War took place when Confederate sympathizers in Baltimore, Maryland, attacked members of the Massachusetts militia en route to Washington, D.C. https://en.wikipedia.org/wiki/Baltimore_riot_of_1861

1971:

The Doors released L.A. Woman, their final album during their lead vocalist Jim Morrison's lifetime. https://en.wikipedia.org/wiki/L.A._Woman

1984:

"Advance Australia Fair", written by Scottish-born composer Peter Dodds McCormick, officially replaced "God Save the Queen" as Australia's national anthem. https://en.wikipedia.org/wiki/Advance_Australia_Fair

2000:

Air Philippines Flight 541 crashed in Samal, Davao del Norte, killing all 131 people on board. https://en.wikipedia.org/wiki/Air_Philippines_Flight_541

2020:

A series of attacks in Nova Scotia, Canada, ended when the perpetrator was killed by police, leaving 22 victims dead. https://en.wikipedia.org/wiki/2020_Nova_Scotia_attacks

---

Wiktionary's word of the day:

cromlech: 1. (archaeology) 2. Synonym of stone circle (“a prehistoric monument consisting of standing stones arranged in a circle”), especially one located in Brittany, France. 3. (UK, chiefly Wales) synonym of altar tomb or dolmen (“a prehistoric megalithic tomb consisting of a (somewhat) flat capstone lying horizontally on two or more upright stones”), especially one located in Wales, but also in England (chiefly Cornwall and Devon) or Ireland. 4. About Word of the Day 5. Nominate a word 6. Leave feedback https://en.wiktionary.org/wiki/cromlech

---

Wikiquote quote of the day:

” --Peter Chung https://en.wikiquote.org/wiki/Peter_Chung